

AgRefactor: Self-Evolving Agentic Workflow for HLS Compatibility and Performance

Anonymous Authors

Abstract—High-Level Synthesis (HLS) provides a fast path from concepts to silicon, but converting real-world software into synthesizable HLS code remains challenging due to restrictive language support and the gap between software and hardware programming practices. Existing automated and LLM-based refactoring approaches partially address this problem, yet they often lack flexibility, struggle to scale, and incur high computational costs. We introduce AGREFACTOR, an LLM-based multi-agent workflow for refactoring software into HLS-compatible programs. AGREFACTOR incorporates a self-evolving memory system that accumulates and retrieves factual and strategic knowledge across tasks, improving robustness and efficiency on unseen programs. To reduce cost and enhance scalability, it integrates automated refactoring tools, enabling agents to balance LLM-driven rewrites with efficient tool-based transformations. On 9 out of 11 challenging real-world benchmarks, which are $5 - 10\times$ longer than the most complex cases studied in prior work, AGREFACTOR outperforms or matches the state-of-the-art automated refactoring tool and a strong LLM-based baseline built on the same framework backbone. Further agentic performance optimization yields a $6.51\times$ geometric mean speedup over the SoTA pragma tuning tool and a $1.20\times$ speedup over optimized open-source designs with less than 20% extra resources. AGREFACTOR is fully-automated, easy to reproduce, and will be open-sourced upon acceptance.

I. INTRODUCTION

High-Level Synthesis (HLS) [1], [2], [3], [4], [5], [6], [7] was introduced to streamline the process of turning design concepts into silicon implementations, by raising the abstraction level from register-transfer level (RTL) design to C/C++-based HLS designs. Practical HLS design flows begin with a tedious step: refactoring the software repository into an HLS-compatible program [8], [9], [10]. As reported in a real-world acceleration project of high energy physics [11], this preparatory refactoring phase can take domain experts *days* to produce an initial HLS-compatible program for further optimization. This highlights the need for more automation to reduce this labor-intensive task.

To address these difficulties, automated refactoring methodologies [8], [9] have shown promise in converting software programs into HLS-synthesizable code. These approaches typically combine software engineering techniques such as dynamic invariant analysis and rely on fixed templates to rewrite non-synthesizable constructs. The increasing capabilities of large language models (LLMs) in software engineering motivate recent studies on using LLMs for HLS refactoring [12], [10]. These methods typically feature an agentic workflow that identifies non-synthesizable constructs and applies iterative refactoring based on compiler feedback. HLSRewriter [12] introduces retrieval-augmented generation (RAG) to store factual knowledge and rewrite templates, while C2HLSC [10] first extracts each function and performs kernel-level rewriting with automatic testbench generation.

Existing methods face three key challenges in HLS refactoring: generalizability, scalability, and cost. To mitigate these challenges, we propose AGREFACTOR, an LLM-based multi-agent workflow capable of refactoring real-world software programs into HLS-compatible programs. To address the issue of generalizability, we develop a self-evolving workflow with a specialized agentic memory system.

TABLE I
AGREFACTOR ACHIEVES BETTER REFACTORING PERFORMANCE
WHILE REDUCING COST COMPARED WITH CURRENT
STATE-OF-THE-ARTS.

Method	Generalizability	Scalability	Cost
Rule-based [8]	✗ Poor	✓ Good	✓ Good
LLM-based [12]	○ Medium	○ Medium	✗ Poor
AGREFACTOR	✓ Good	✓ Good	○ Medium

Factual and strategic knowledge is automatically extracted and retrieved as the agents interact with diverse tasks (programs) and HLS tools. As the system accumulates knowledge from past experience, AGREFACTOR demonstrates higher efficiency and robustness when applied to unseen software programs. Unlike manually crafted RAG approaches, our memory system adapts to the model’s specific failure and success modes, providing targeted knowledge retrieval that improves refactoring accuracy and efficiency. Relative to general agentic memory systems [13], [14], [15], [16], [17], [18], we create a specialized knowledge format, dedicated interaction patterns, and a novel distance metric, all designed to achieve superior performance on this task.

To alleviate the challenges of scalability and cost, we leverage the tool-calling capability of LLM agents. Specifically, we equip AGREFACTOR with HeteroRefactor [8] (abbreviated as HeteroRF in figures and tables), the state-of-the-art (SoTA) automated open-source refactoring tool for HLS. Naturally, for programs that can be fully handled by tools, we avoid the computational overhead of invoking LLMs. Furthermore, our framework allows LLM agents to balance between code generation and tool calls: they focus on removing or rewriting software constructs unsupported by HeteroRefactor, while delegating the remaining work to the tool. As a result, our tool-enhanced agentic workflow combines the flexibility of LLM-based code rewriting with the scalability and low cost of automated refactoring.

While recent search-based approaches like AlphaEvolve [19] utilize scalar feedback (e.g., latency) to drive code mutations, AGREFACTOR provides the agent with richer information from raw EDA logs and scheduling reports. This capability allows the agent to pinpoint specific critical-path bottlenecks. Furthermore, we integrate a multi-tool environment that enables the agent to bootstrap from established human knowledge rather than performing blind mutations.

Our contributions are summarized as follows:

- 1) We design and implement AGREFACTOR, an agentic workflow capable of refactoring real-world software into synthesizable, high-performance HLS code. Unlike existing approaches that focus solely on correctness and pragma insertion, we distinctly address structural refactoring, which most significantly impacts hardware performance.
- 2) We propose a self-evolving agent memory system specialized for this task. By continuously accumulating and retrieving both factual and strategic knowledge across varying tasks, our workflow enhances the base LLM’s robustness in HLS refactoring

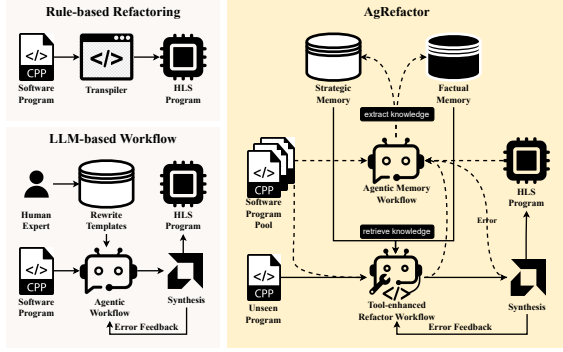


Fig. 1. Limitations of existing methods motivate our workflow design. (1) Automated refactoring suffer from poor generalizability. State-of-the-art automated refactoring cannot handle external libraries and complex struct data types. (2) LLM-based workflows require manual effort to build a RAG system and are subject to the intrinsic randomness of LLMs, leading to high variance in outputs. Moreover, pure LLM-based refactoring incurs significant cost. In contrast, AGREFACTOR addresses these challenges through a tool-enhanced, self-evolving workflow, enabled by an agentic memory system (Sec. III-A).

while requiring minimal human intervention.

- 3) We integrate automated refactoring tools into our agentic framework, creating a hybrid pipeline powered by both LLM agents and an algorithmic backend. This integration significantly reduces computational costs and improves the scalability of LLM-driven HLS refactoring.

Evaluation on diverse real-world cases shows AGREFACTOR outperforms or matches both the state-of-the-art automated refactoring tool and a strong LLM-based baseline on 9 out of 11 benchmarks. Our optimization agent achieves a $6.51\times$ geometric mean speedup over SoTA pragma tuning tools AutoDSE [20] and a $1.2\times$ speedup over highly optimized open-source HLS designs with less than 20% extra resource.

II. MOTIVATION

In this section, we motivate our workflow design by evaluating existing approaches with a comprehensive benchmark and highlighting their limitations. Existing approaches are usually evaluated on a small number of manually constructed test cases — typically ones that can be readily solved by their framework. To more thoroughly examine the limitations of state-of-the-art approaches, we evaluate them on an extended set of real-world benchmarks, which includes multiple standalone files with up to 1000 lines of code. The details of our benchmark suite are provided in Table II and Section III-D. Since existing workflows are not fully open-sourced and are tailored specifically to Catapult HLS tools, we make our best effort to reproduce their approach.

Figure 2 summarizes the results. Although both HeteroRefactor and HLSRewriter perform well on their own benchmarks, they failed many more larger benchmarks, such as libjpeg-turbo. Beyond the known limitation of handling external libraries such as STL containers (e.g., “std::set” and “std::map”), HeteroRefactor also shows limitations when dealing with common pointer operations. In particular, it cannot process nested pointer arithmetic ($A \rightarrow B \rightarrow C$) correctly.

State-of-the-art LLM workflows, even those backed by GPT-5, show limited success on real-world repositories (Fig. 2). Three main challenges emerge: (1) real-world programs contain incompatible

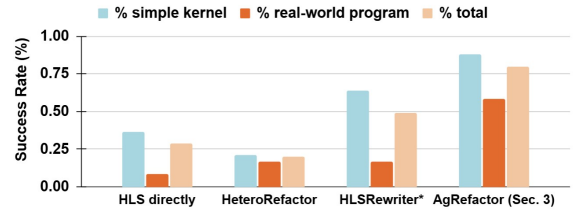


Fig. 2. Summary of one-shot refactoring success rate across different workflows. We adapt the HLSRewriter methodology onto our framework as a controlled ablation (see Section IV-A details). Each baseline is tested with 45 kernels. The category “real-world program” includes amalgamated files from real-world software repositories together with the “strassen” kernel, while “simple-kernel” covers the remaining benchmarks listed in Table II. All LLM-based workflows use “GPT-5-mini”. Existing methods fall short in refactoring real-world benchmarks.

implementations missing from standard knowledge bases, (2) refactoring fails when invalid patterns span multiple locations in long programs, and (3) iterative error feedback makes LLMs slow and computationally expensive compared to automated tools. Furthermore, our analysis reveals LLMs frequently misuse unsupported C++ features (e.g., lambda functions) or incorrectly expose internal structs to hardware interfaces.

Because LLMs make random mistakes, anticipating them via manual prompt engineering requires impractical effort. Instead, robust refactoring requires a task-specific plan that properly decomposes the process. To this end, we propose a self-evolving agentic workflow that automatically accumulates factual and strategic knowledge from diverse tasks, retrieving it to guide agents on new programs.

Since the original HLSRewriter targets the Catapult HLS toolchain and its key components (the RAG knowledge base and identifier prompts) are not open-sourced, a faithful reproduction is infeasible. We therefore construct a strong baseline that adapts the HLSRewriter methodology onto our own framework, as detailed in Section IV-A.

III. METHODOLOGY

A. Framework

Figure 3 shows the overall workflow of AGREFACTOR. The user provides a C/C++ program and specifies a top-level function; from that point on, the framework is fully automated.

The pipeline begins with a *Test Generator* that creates tests and interface assumptions (e.g., array sizes and function signatures) before refactoring, since the testbench must call the refactored top-level function. These assumptions propagate as high-level constraints to all downstream stages. Next, a two-stage diagnosis and planning pipeline identifies non-synthesizable constructs: category-specific *Identifiers* run in parallel, an *Item Organizer* merges and deduplicates their findings, and a *Planner* produces a complete refactoring plan, strengthened by strategic and factual memory (Section III-B). A *Refactor Worker* then applies this plan, and the result is validated through synthesis and simulation. If errors arise, an *Analyzer-Fixer* pair iteratively updates the code or testbench until both stages succeed or a retry limit is reached. Each agent is initialized with a role-specific system prompt. The *Identifiers* and *Planner* are further enhanced with long-term memory, so that their context is progressively enriched.

B. Long-term Memory for HLS Refactoring

To enable self-evolution, AGREFACTOR accumulates successful and unsuccessful trials as queryable knowledge. Each memory entry is defined as a tuple (p_i, I_i, s_i, c_i) , where p_i is the initial program,

TABLE II

A COMPREHENSIVE BENCHMARK SUITE WITH COMPLEX REAL-WORLD EXAMPLES. OUR BENCHMARKS COVER REAL-WORLD PROGRAMS FROM IMAGE/VIDEO CODECS, CRYPTOGRAPHY, AND COMPUTATIONAL GENOMICS.

Category	Name	Lines of code
leetcode [21]	maxSlidingWindow, delNodes, wordBreak, getSkyline, canDistribute criticalConnections, smallestRange, solveSudoku, removeInvalidParentheses, swimInWater	21-67
HLSRewriter [12]	regUpdate, reverseKGroup, trap, FibSubseq, knn, rotateGrid, maxPoints, long, qrd_compute, forward, dfs, aes_encrypt	10-115
C2HLS [10]	Frequency, CumulativeSums, Overlapping, quicksort, present80_encryptBlock, sha256_update, des_crypt	11-221
HeteroRF [8]	dfs, mergesort, linkedlist, ahocorasick, strassen	65-304
libsodium [22]	blake2b_compress, chacha20_stream, argon2_fill_segment	126-370
minimap2 [23]	mm_chain_dp_orig	473
libjpeg-turbo [24]	encode_one_block, median_cut, select_colors, fill_inverse_cmap, idct_generic	279-754
av1 [25]	av1_apply_temporal_filter, av1_compound_type_rd	407-1266

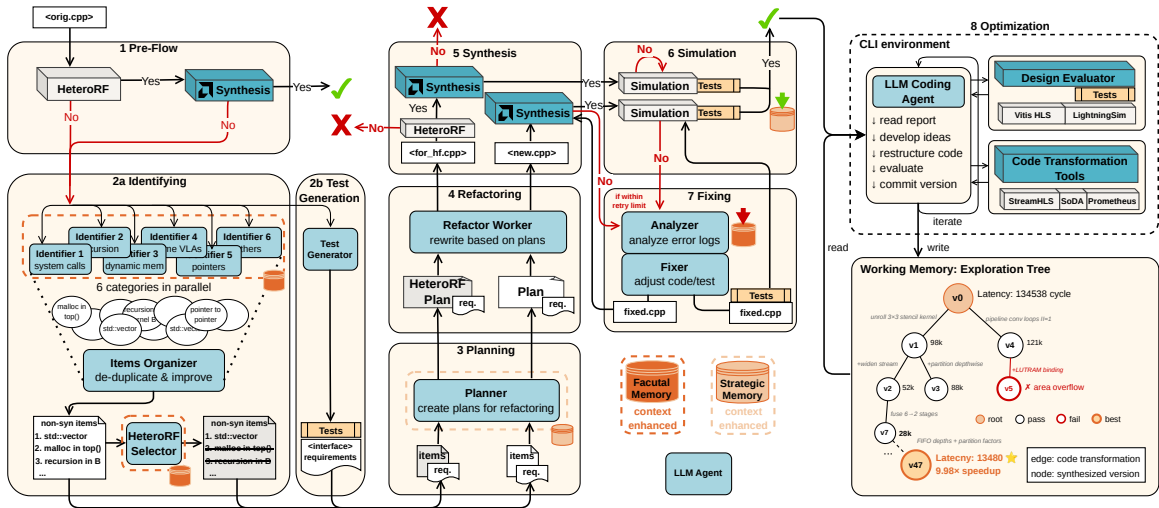


Fig. 3. Overview of AGREFACTOR. Given a C/C++ program and a user-specified top-level function, the framework automatically produces a synthesizable HLS implementation by progressing through identifying, planning, refactoring, and fixing stages, while continuously updating a long-term memory bank. Once refactored, the synthesizable code and its testbench are forwarded to a performance optimization agent. Situated in a CLI environment with access to evaluation and code transformation tools, this agent leverages an internal tree-based working memory to systematically explore, apply, and track diverse code optimization strategies for optimal performance.

<Analyzer>		
#1: emphasizing known issues	#2: guiding feasible pragma insertion	#3: customizing instructions for common mistakes of the current LLM
1. Using b1/b2 pointer aliases across boxlist.	3. Overaggressive pipelining implies unintended complete unrolling.	5. Unbounded array parameter 'boxlist[]' complicates pointer analysis.
2. Returning pointers to array elements is not synthesizable.	4. Applied <code>ap_none</code> to non-scalar ports 'img' and 'colormap'.	6. Using <code>NULL</code> in C++ instead of <code>nullptr</code> .
		7. Function duplication due to inconsistent call-site properties.
<Test Generator>		<Identifiers>
- New top signature to implement: int encode_one_block_hls(const JOEUF block[DCTSIZE2], int last_dc_val...);		The identified non-synthesizable item list: {
- MACROs/typedefs to keep: #define DCTSIZE2 64 #define MAX_COMPS_IN_SCAN 4 #define BUFSIZE (DCTSIZE2 * 8)		0. state: struct pointer
- I/O contract: - Read initial bit-buffer state from 'io_put_buffer' and 'io_free_bits'; update them with final state.		1. error_exit: function-pointer
- Output rules: - Apply JPEG byte-stuffing (emit 0x00 after any 0xFF byte).		2. memcpy: standard library call
		3. error_exit(...): ...
		...
<Items Organizer>		
{"index_to_remove": [3, 5, 8], "other_necessary_items": []}		

Fig. 4. Messages passed between agents.

I_i lists the identified incompatible constructs, s_i is the refactoring strategy, and c_i is the generalized critique generated by the *Analyzer*. During the memory accumulation phase on a diverse training set $\mathcal{P}$, the *Analyzer* derives these critiques from simulation or synthesis logs. To ensure these insights are broadly applicable to unseen programs, we instruct the *Analyzer* to omit program-specific variables or function names. To guarantee compliance and reliability, we enforce a lightweight validation step where the *Analyzer*'s output is cross-referenced against the original variable and function names. Critiques containing exact matches are automatically rejected and regenerated. For successful trials, we simply record the program, plan, and identified items.

When approaching a new task $q \notin \mathcal{P}$, the workflow retrieves relevant prior knowledge using a specialized distance metric that captures both global structural similarity and local implementation details. Relying solely on code embeddings compresses too much detail and may overlook non-synthesizable constructs. Thus, we combine the distance between the raw program embeddings (e^c) and the identified item embeddings (e^i):

$$d(q \notin \mathcal{P}, p \in \mathcal{P}) = \alpha|e_q^c - e_p^c|_2 + (1 - \alpha)|e_q^i - e_p^i|_2$$

Using this metric, we retrieve the plan s_i from the most similar successful trials to augment the Planner’s context, and the items I_i from similar failed trials to sharpen the Identifiers’ detection capabilities.

C. Tool Integration

To mitigate the high computational costs of LLMs, we integrate our framework with HeteroRefactor. Because its runtime is negligible (<5s), we adopt a greedy strategy: we first attempt automated refactoring with HeteroRefactor, terminating the flow immediately upon success.

While the tool alone lacks generalizability (Section II), many of its limitations are resolvable through lightweight code rewrites. If the initial attempt fails, a dedicated *Tool Specialist* pipeline is triggered. The original program and identified unsupported items are passed to the *Planner* and *Refactor Worker* to perform tractable modifications (e.g., removing problematic headers). This targeted intervention makes the code “HeteroRefactor-compatible,” significantly improving overall coverage and efficiency while avoiding the overhead of full LLM-driven generation.

D. Task Creation

To construct the program set $\mathcal{P}$ for memory accumulation, we adapt existing benchmarks to Vitis HLS and extract new, standalone programs from real-world software repositories. To overcome the structural complexity of these repositories, we deploy an LLM-based agent that automates benchmark generation through four steps: selecting diverse candidate functions to maximize the coverage of non-synthesizable constructs, generating state-initializing wrappers, extracting standalone code, and validating functional correctness against the original implementation.

E. Performance Optimization

Beyond synthesizability, we develop an optimization agent that iteratively improves hardware performance. The agent operates in a CLI environment integrating C-based simulation for correctness, Vitis HLS for synthesis, and LightningSim [26] for fast latency estimation—bypassing time-consuming RTL simulation entirely. It also has access to multiple source-to-source code transformation tools [4], [27]. These tools are wrapped behind a simplified command-line interface to stabilize tool calling and reduce EDA scripting effort.

The agent’s system prompt enforces a strict three-phase optimization hierarchy: (1) structural rewrites first, (2) scheduling-report analysis to resolve critical-path violations, and (3) fine-grained pragma tuning last. This prevents the common failure mode of naive LLM-based generators that randomly apply pragmas without first establishing a sound architecture.

To keep the agent effective over multi-hour runs, we develop a tree-based working memory pinned to its context. Each node in the tree represents a synthesized HLS code version, and the agent constructs links between nodes via condensed summaries of its optimization strategies and synthesis metrics. As the agent optimizes freely, we actively monitor both its evaluation results and context usage. To prevent context degradation, our programmatic monitors track the context window size and trigger the agent to update this tree periodically, especially before context compaction occurs. If these monitors detect performance stagnation, such as repeated simulation failures or a latency plateau, they redirect the agent to a knowledge base of successful past strategies and structural templates. This keeps

the working memory compact and prevents the agent from getting stuck in unproductive loops.

IV. EVALUATION

We implement AGREFACTOR using the AG2 framework [28]. For memory storage, we rely on ChromaDB to record both successful and failed trials, and we compute embeddings for distance measurement and retrieval with the “all-MiniLM-L6-v2” model from the Sentence-Transformer [29] package.

A. Experiment Setup

Our baselines for comparison are: (1) HeteroRefactor [8], the state-of-the-art automated refactoring tool, and (2) an HLSRewriter-style baseline [12] built on top of our own framework. Because the original HLSRewriter targets the Catapult HLS toolchain and its key components, such as the curated RAG knowledge base and identifier prompts, are not open-sourced, a direct reproduction is infeasible. Instead, we construct this baseline as a controlled ablation of AgRefactor: we replace the self-evolving memory with the official Vitis HLS programming guide as a static RAG source, collapse the six category-specific Identifiers into a single general-purpose Identifier, and remove the Tool Specialist. Critically, this baseline retains all other infrastructure: the fixing loop, synthesis/simulation pipeline, test generator, and Planner. This makes it stronger than a naive reimplementation and ensuring that observed gains are attributable to our proposed components. We further ablate AGREFACTOR by comparing (i) AGREFACTOR with no memory augmentation, (ii) AGREFACTOR with only memory augmentation, and (iii) AGREFACTOR with both memory augmentation and tool integration.

The benchmark suite contains 45 cases, which we split into 24 for accumulating experience and 21 for testing the memory-augmented flow. We run the outer loop of memory accumulation for three epochs on the 24 training cases. During testing, each LLM-based workflow is executed 20 times per benchmark, and we report both the success rate and the average number of retries within the inner fixing loop. Because of the high cost of repeated evaluations, we exclude certain cases where a simple iterative flow already achieves 100% success rates (Figure 2) and focus our evaluation on the more challenging benchmarks. We set $\alpha = 0.6$ in all experiments.

We tested the GPT family of models. For the inner fixing flow, we cap the maximum number of iterations at three. In the memory-enabled workflow, we retrieve one plan from the most similar past task based on the distance metric defined in Section III-B, and we retrieve critiques from the top three most similar programs to enhance the context for the Identifiers. All models are run with the default parameters provided by the respective LLM APIs.

B. Comparison with Baselines

Table III summarizes the results of comparing AGREFACTOR with HeteroRefactor and HLSRewriter. The improvements over HLSRewriter can be classified into three categories: (1) benchmarks that can be directly refactored by HeteroRefactor, (2) cases where the combined effort of the *Identifiers*, *Planners*, and memory augmentation improves accuracy on challenging benchmarks, and (3) cases where the *Tool Specialist* rewrites constructs unsupported by HeteroRefactor, enabling HeteroRefactor to generate a synthesizable version through correct-by-construction transformations.

Examples of category (1) include test cases “ahocorasick,” “dfs,” and “strassen,” where AGREFACTOR’s tool-enhanced flow achieves the same results as HeteroRefactor with no LLM invocation. For 8 out of 11 benchmarks, our memory mechanism increases the success

rate, with further details provided in Section IV-C1. For test cases “encode_one_block” and “idct_generic,” the *Tool Specialist* successfully rewrites the code to match the capabilities of HeteroRefactor while maintaining functional equivalence.

On two benchmarks, “median_cut” and “argon2_fill_segment”, the HLSRewriter-style baseline slightly outperforms the full AgRefactor pipeline. Since both configurations share the same framework, this indicates that on certain benchmarks where a static knowledge source (the Vitis manual) already covers the relevant constructs, memory retrieval from dissimilar past tasks can introduce marginal noise.

TABLE III

COMPARISON BETWEEN AGREFACTOR AND CURRENT SOTAS. EACH BENCHMARK IS RUN 20 TIMES, AND WE REPORT THE NUMBER OF SUCCESSFUL RUNS. OVERALL, AGREFACTOR OUTPERFORMS OR MATCHES SoTA APPROACHES ON 9 OUT OF 11 BENCHMARKS.

Task (N=20)	HeteroRF	HLSRewriter	Ours
av1_compound_type_rd	0	0	1
encode_one_block	0	2	10
idct_generic	0	2	5
median_cut	0	20	18
argon2_fill_segment	0	17	13
mm_chain_dp_orig	0	5	6
ahocorasick	20	16	20
dfs	20	16	20
strassen	20	17	20
wordbreak	0	17	20
skyline	0	20	20

Even for simple cases (“ahocorasick” to “skyline”), where a basic iterative workflow already achieves high accuracy, AGREFACTOR’s memory mechanism further improves performance, reaching nearly 100%. Manual inspection suggests this is because similar benchmarks appear in the memory pool, making retrieval particularly effective when the gap between “training” (used for memory accumulation) and “testing” benchmarks is small.

This also explains why improvements are limited on larger programs such as “av1_compound_type_rd”. With over 1,000 lines of code, it introduces a significant distribution shift relative to stored tasks, making past strategies and critiques less transferable.

Overall, our results highlight the value of leveraging past experience through a self-evolving agentic workflow.

C. Ablation Study

In this section, we ablate two core components of AGREFACTOR: the memory mechanism and the tool specialist. The results highlight how these enhancements extend beyond a fixed agentic workflow, providing clear advantages in both accuracy and robustness.

1) *Ablating Memory Augmentation*: Table IV and Table V report the results of AGREFACTOR with and without the proposed memory mechanism, using either the weaker “GPT-5-mini” or the stronger “GPT-5” as the base LLM. Across both models, we observe a consistent trend: adding memory augmentation improves the refactoring success rate. With GPT-5-mini, the memory-enabled workflow outperforms the no-memory baseline on 10 out of 11 benchmarks. With GPT-5, we see accuracy gains on all benchmarks overall, and the memory-enabled workflow outperforms or matches the baseline on 4 out of 6 benchmarks, showing a particularly large improvement on the “encode_one_block” benchmark.

Beyond accuracy, memory augmentation also reduces the number of retries (i.e., inner fixing iterations), such as “ahocorasick”, “dfs” and “word_break”. This aligns with the intuition that specialized instructions stored in memory improve the robustness of the base

TABLE IV
ABLATION STUDY OF AGREFACTOR WITH AND WITHOUT MEMORY AUGMENTATION, USING GPT-5-MINI AS THE BASE LLM.

Task (N=20)	no-mem		with-mem	
	# pass	# retry	# pass	# retry
av1_compound_type_rd	0	0.0	1	1.0
encode_one_block	3	1.0	5	1.4
idct_generic	3	0.3	5	1.6
median_cut	13	0.4	18	0.4
argon2_fill_segment	12	0.8	13	1.3
mm_chain_dp_orig	9	0.3	6	0.7
ahocorasick	17	0.3	18	0.4
dfs	18	0.4	20	0.1
strassen	17	0.4	19	0.1
wordbreak	17	1.1	20	1.0
skyline	20	0.3	20	0.6
% Winning (v.s. no-mem)				82%
% Draw				0%

TABLE V

ABLATION STUDY OF AGREFACTOR WITH AND WITHOUT MEMORY AUGMENTATION, USING GPT-5 AS THE BASE LLM.

Task	GPT-5 (N=20)	
	no-mem (# pass)	with-mem (# pass)
av1_compound_type_rd	6	5
encode_one_block	2	9
idct_generic	0	2
median_cut	15	18
argon2_fill_segment	14	18
mm_chain_dp_orig	18	18
% Winning (v.s. no-mem)		67%
% Draw		10%

LLM. Note that we only record the number of retry if the synthesis and simulation succeed in the end; therefore, it is possible to have larger number of retry when the success rate improves.

2) *Ablating Tool Calling*: To isolate the effect of the *Tool Specialist*, we conduct an ablation study comparing two settings: running HeteroRefactor alone and running HeteroRefactor with the *Tool Specialist*. The results, shown in Table VII, demonstrate that adding the *Tool Specialist*, which performs simple rewrites, significantly improves success rates on “encode_one_block”, “idct_generic” and “argon2_fill_segment”. A promising future direction is to enable LLM-based agents to directly modify or extend the capabilities of the tool itself.

D. Refactoring for Optimization

The synthesizable designs produced by AGREFACTOR serve as starting points for performance optimization. We compare two approaches under a 2-hour budget: (1) AutoDSE [20], which explores pragma configurations (PIPELINE, UNROLL, TILE) without modifying the code structure, and (2) AGREFACTOR’s optimization agent (GPT-5.3-codex) augmented with code-transformation tools. As shown in Table VIII, the AGREFACTOR optimization agent achieves a geometric mean speedup of $6.51\times$ over AutoDSE alone. These gains stem from structural transformations inaccessible to pragma-only DSE: replacing an $O(n^2)$ sort with a radix sort (skyline, $295\times$), restructuring a DFS stack (word_break, $29\times$), and eliminating runtime table initialization (idct_generic, $10\times$). Because the agent

TABLE VI
EXTENDED HLS OPTIMIZATION BENCHMARK SUITE (51 DESIGNS).

Source	Name	Lines of code
StreamHLS [4] (14)	atax, bicg, gemm, gesummv, mvt, k2mm, k3mm, k7mm $\times$ 2, Autoencoder, ResidualBlock, ResMLP, DepthwiseConv, MultiHeadSelfAttention	127-990
Prometheus [27] (9)	atax, bicg, gemm, gemver, gesummv, mvt, symm, 2mm, 3mm	360-1050
SoDA [30] (6)	jacobi2d, jacobi3d, heat3d, seidel2d, sobel2d, denoise2d	628-1853
HLSFactory [31] (6)	3d-rendering, optical-flow, SkyNet, ECG_Classifier, GP_fixed, spam-filter	334-2290
Vitis Libraries (L2) [32] (16)	jpeg_decoder, resize, lz4 $\times$ 2, snappy $\times$ 2, geoip, kmeans, black_scholes, swaption, portfolio, gematrixinverse, geqrf, getrf $\times$ 2, trtrs	140-2286

TABLE VII
ABLATION STUDY OF HETEROREFACTOR WITH AND WITHOUT A *Tool Specialist*, USING GPT-5-MINI AS THE BASE LLM.

Task	GPT-5-mini (N=20)	
	no-specialist	with-specialist
av1_compound_type_r	0	0
encode_one_block	0	10
idct_generic	0	5
median_cut	0	0
argon2_fill_segment	0	8
mm_chain_dp_orig	0	0
ahocorasick	20	20
dfs	20	20
strassen	20	20
wordbreak	0	0
skyline	0	0

is instructed to utilize the available pragma tuning tools initially, it matches AutoDSE’s performance on benchmarks where pragma parallelism dominates (*compound_type_rd*, *strassen*). Furthermore, running AutoDSE for 8 hours yields performance identical to its 2-hour run, underscoring the critical importance of structural refactoring for performance optimization.

TABLE VIII
OPTIMIZATION COMPARISON ON AGREFACTOR DESIGNS, NORMALIZED TO AUTODSE. BOTH APPROACHES WERE RUN FOR 2 HOURS ON THE SAME REFACTORED CODE. DESIGNS THAT AUTODSE FAILED TO PROCESS ARE EXCLUDED.

Benchmark	AutoDSE	AgRefactor
compound_type_rd	1.00 $\times$	1.00 $\times$
strassen	1.00 $\times$	1.00 $\times$
skyline	1.00 $\times$	295.44 $\times$
word_break	1.00 $\times$	28.71 $\times$
idct_generic	1.00 $\times$	9.95 $\times$
encode_one_block	1.00 $\times$	6.30 $\times$
ahocorasick	1.00 $\times$	4.34 $\times$
mm_chain_dp	1.00 $\times$	1.40 $\times$
Geomean	1.00 $\times$	6.51$\times$

We further evaluate the tool-integrated iterative optimization capability on 51 open-source HLS benchmarks spanning five categories (Table VI). We instruct the agent to invoke code transformation tools and then perform free-form iterative code refinement for 2 hours. During evaluation, we target the Xilinx U200 device, with the exception of the Prometheus benchmarks, which utilize the U55C setting to maintain consistency with prior work [27]. Figure 5 reports the geometric mean speedup by benchmark category

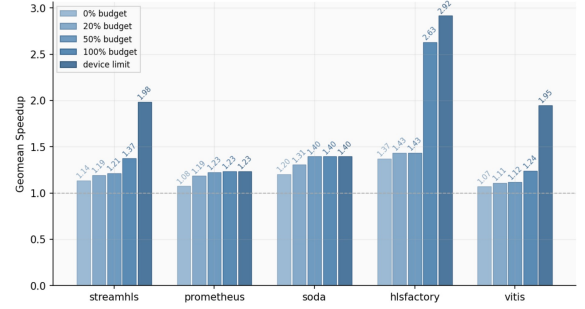


Fig. 5. Performance improvement over optimized open-source designs under different extra resource budgets. Device limits are Xilinx U200 (U55C for Prometheus).

under increasing extra resource budgets. With no additional resource budget allowed (0%), the agent achieves a 1.14 $\times$ geometric mean speedup through code restructuring alone. The techniques discovered by the agent include pure algorithmic rewrites, HLS-level register insertion, and resource binding. Relaxing the budget to 20%, 50%, and 100% additional resources enables progressively more aggressive area/latency tradeoffs, reaching speedups of 1.20 $\times$, 1.23 $\times$, and 1.41 $\times$, respectively. The HLSFactory benchmarks benefit the most from relaxed budgets (1.37 $\times$ $\rightarrow$ 2.63 $\times$), as certain baselines are largely unoptimized and do not exploit sufficient parallelism.

E. Runtime and Cost

The runtime of our workflow depends on the LLM serving setup. (1) With commercial APIs, AGREFACTOR without tools already runs efficiently: 3-20 minutes (avg. 10), over 10x faster than manual refactoring, and only 9% slower than HLSRewriter. (2) The tool-enabled flow further improves speed, averaging 4 minutes versus 10. (3) Tool-calling agents also reduce cost, as tool usage incurs no API token overhead.

V. CONCLUSION

We present AGREFACTOR, an agentic approach for refactoring software into HLS-compatible implementations. Our method introduces a self-evolving memory mechanism based on trial-and-error and integrates a rule-based refactoring tool to mitigate the high computational cost of LLM inference. Experimental results demonstrate superior refactoring accuracy and efficiency compared with both algorithmic and prior LLM-based methods.

REFERENCES

- [1] J. Cong, B. Liu, S. Neuendorffer, J. Noguera, K. Vissers, and Z. Zhang, “High-level synthesis for FPGAs: From prototyping to deployment,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 30, no. 4, pp. 473–491, 2011.
- [2] J. Cong, J. Lau, G. Liu, S. Neuendorffer, P. Pan, K. Vissers, and Z. Zhang, “FPGA HLS today: successes, challenges, and opportunities,” *ACM Transactions on Reconfigurable Technology and Systems (TRETS)*, vol. 15, no. 4, pp. 1–42, 2022.
- [3] H. Ye, C. Hao, J. Cheng, H. Jeong, J. Huang, S. Neuendorffer, and D. Chen, “ScaleHLS: A new scalable high-level synthesis framework on multi-level intermediate representation,” in *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2022, pp. 741–755.
- [4] S. Basalama and J. Cong, “Stream-HLS: Towards automatic dataflow acceleration,” *arXiv e-prints*, pp. arXiv–2501, 2025.
- [5] H. Ye, H. Jun, and D. Chen, “HIDA: A hierarchical dataflow compiler for high-level synthesis,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, 2024, pp. 215–230.
- [6] S. Pouget, L.-N. Pouchet, and J. Cong, “A unified framework for automated code transformation and pragma insertion,” in *Proceedings of the 2025 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*, 2025, pp. 187–198.
- [7] H. Chen, N. Zhang, S. Xiang, Z. Zeng, M. Dai, and Z. Zhang, “Allo: A programming model for composable accelerator design,” *Proceedings of the ACM on Programming Languages*, vol. 8, no. PLDI, pp. 593–620, 2024.
- [8] J. Lau, A. Sivaraman, Q. Zhang, M. A. Gulzar, J. Cong, and M. Kim, “Heterorefactor: refactoring for heterogeneous computing with fpga,” in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, 2020, pp. 493–505.
- [9] Q. Zhang, J. Wang, G. H. Xu, and M. Kim, “Heterogen: transpiling c to heterogeneous hls code with automated test generation and program repair,” in *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2022, pp. 1017–1029.
- [10] L. Collini, S. Garg, and R. Karri, “C2HLS: Can LLMs bridge the software-to-hardware design gap?” *arXiv preprint arXiv:2406.09233*, 2024.
- [11] A. Wojenski, H. Zbroszczyk, M. Kruszewski, P. Szymanski, E. Wawrzyn, D. Wielanek, W. Zabolotny, D. Pawlowska, and T. Gniazdowski, “Hardware acceleration of complex HEP algorithms with HLS and FPGAs: Methodology and preliminary implementation,” *Computer Physics Communications*, vol. 295, p. 108997, 2024.
- [12] K. Xu, G. L. Zhang, X. Yin, C. Zhuo, U. Schlichtmann, and B. Li, “Hlsrewriter: Efficient refactoring and optimization of c/c++ code with llms for high-level synthesis,” *ACM Transactions on Design Automation of Electronic Systems*, 2025.
- [13] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” *arXiv preprint arXiv:2005.11401*, 2020.
- [14] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, “Memgpt: Towards llms as operating systems,” *arXiv preprint arXiv:2310.08560*, 2023.
- [15] B. Jiménez Gutiérrez, Y. Shu, Y. Gu, M. Yasunaga, and Y. Su, “Hipporag: Neurobiologically inspired long-term memory for large language models,” *arXiv preprint arXiv:2405.14831*, 2024.
- [16] Z. Z. Wang, J. Mao, D. Fried, and G. Neubig, “Agent workflow memory,” *arXiv preprint arXiv:2409.07429*, 2024.
- [17] Y. Wang, Y. Gao, X. Chen, H. Jiang, S. Li, J. Yang, Q. Yin, Z. Li, X. Li, B. Yin, J. Shang, and J. McAuley, “MemoryLLM: Towards self-updatable large language models,” *arXiv preprint arXiv:2402.04624*, 2024.
- [18] W. Xu, Z. Liang, K. Mei, H. Gao, J. Tan, and Y. Zhang, “A-MEM: Agentic memory for LLM agents,” *arXiv preprint arXiv:2502.12110*, 2025.
- [19] A. Novikov, N. Vū, M. Eisenberger, E. Dupont, P.-S. Huang, A. Z. Wagner, S. Shirobokov, B. Kozlovskii, F. J. Ruiz, A. Mehrabian *et al.*, “Alphaevolve: A coding agent for scientific and algorithmic discovery,” *arXiv preprint arXiv:2506.13131*, 2025.
- [20] A. Sohrabzadeh, C. H. Yu, M. Gao, and J. Cong, “AutoDSE: Enabling software programmers to design efficient FPGA accelerators,” *ACM Trans. Des. Autom. Electron. Syst.*, vol. 27, no. 4, Feb. 2022. [Online]. Available: <https://doi.org/10.1145/3494534>
- [21] LeetCode, “Leetcode,” 2025, accessed: 2025-10-02. [Online]. Available: <https://leetcode.com/>
- [22] F. Denis and the libsodium contributors, “libsodium,” 2025, accessed: 2025-10-02. [Online]. Available: <https://doc.libsodium.org/>
- [23] H. Li, “minimap2,” 2025, accessed: 2025-10-02. [Online]. Available: <https://github.com/lh3/minimap2>
- [24] The libjpeg-turbo Project, “libjpeg-turbo,” 2025, accessed: 2025-10-02. [Online]. Available: <https://libjpeg-turbo.org/>
- [25] Alliance for Open Media, “Av1 reference codec (libaom),” 2025, accessed: 2025-10-02. [Online]. Available: <https://aomedia.googlesource.com/aom/>
- [26] R. Sarkar, R. Paul, and C. C. Hao, “LightningSimV2: Faster and scalable simulation for high-level synthesis via graph compilation and optimization,” in *2024 IEEE 32nd Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, 2024, pp. 104–114.
- [27] S. Pouget, M. Lo, L.-N. Pouchet, and J. Cong, “Holistic Optimization Framework for FPGA Accelerators,” *ACM Transactions on Design Automation of Electronic Systems*, vol. 31, no. 1, pp. 1–37, 2025.
- [28] C. Wang, Q. Wu, and the AG2 Community, “Ag2: Open-source agents for ai agents,” 2024, available at <https://docs.ag2.ai/>. [Online]. Available: <https://github.com/ag2ai/ag2>
- [29] UKPLab, “Sentencetransformers,” 2025, accessed: 2025-10-02. [Online]. Available: <https://github.com/UKPLab/sentence-transformers>
- [30] Y. Chi, J. Cong, P. Wei, and P. Zhou, “Soda: Stencil with optimized dataflow architecture,” in *Proceedings of the International Conference on Computer-Aided Design*, 2018, pp. 1–8.
- [31] S. Abi-Karam, R. Sarkar, A. Seigler, S. Lowe, Z. Wei, H. Chen, N. Rao, L. John, A. Arora, and C. Hao, “HLSFactory: A framework empowering high-level synthesis datasets for machine learning and beyond,” in *Proceedings of the 2024 ACM/IEEE International Symposium on Machine Learning for CAD*, 2024, pp. 1–9.
- [32] AMD/Xilinx, “Vitis Libraries,” https://github.com/Xilinx/Vitis_Libraries, 2024.